SPRINT DEMO

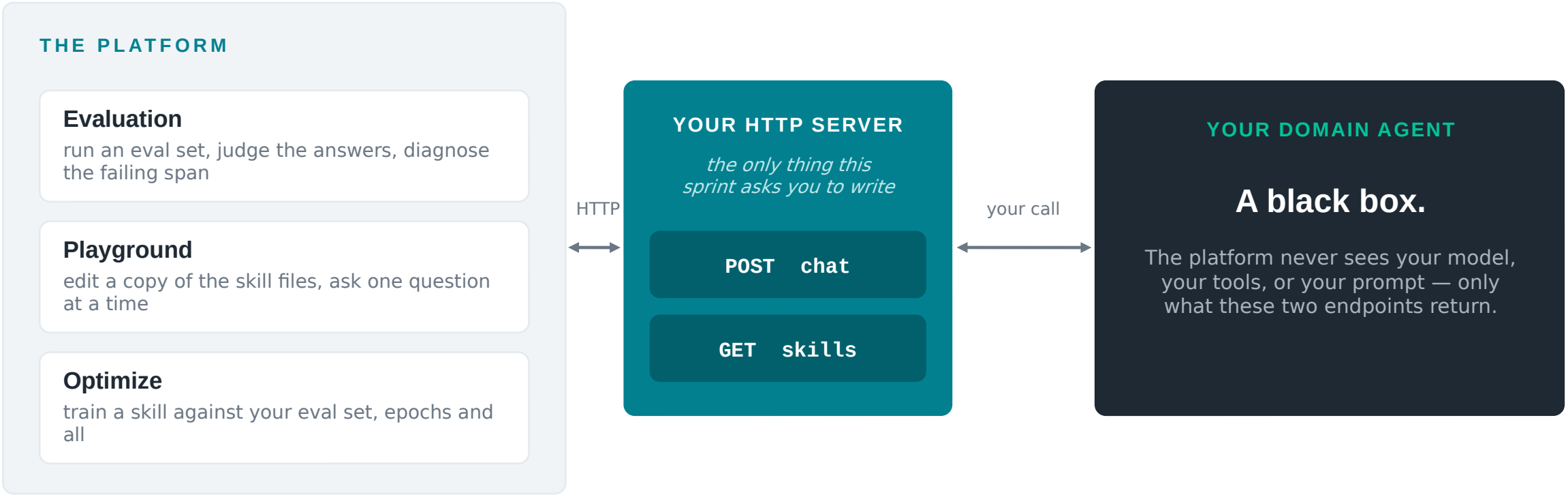
Plugging your domain agent into the platform

Two endpoints, three levels of integration — and what we learned pointing the eval loop at skill selection.

1. The HTTP seam — how your agent connects
2. Evaluating skill selection, and optimizing it

PART 1 · THE SEAM

Where your agent meets the platform



Traces close the loop: your agent writes to Langfuse under the trace id we send, and the platform reads them back.

Everything left of the seam is ours to maintain. Everything right of it stays yours.

PART 1 · THE CONTRACT

Two endpoints — and only the first is required

Two absolute URLs, entered separately — nothing is appended to a base URL, and no path is imposed.

`POST {chat endpoint}`

Answer one question.

- A plain OpenAI chat completions endpoint — if you already have one, you are already integrated
- Single-shot: no conversation, no session, no state between calls
- Everything the platform needs beyond the standard rides in one extra top-level key: skill_studio
- Ignore any key you do not recognise

`GET {skills endpoint}`

List your skill files.

- Optional — without it, evaluation still runs normally
- Returns your whole skill workspace as a flat {path: text} map
- Plus a version string that moves whenever your answers would move
- No parameters, no request body

Authentication unlocks no feature — but the platform can send a credential, in either of two shapes.

A key somebody typed

A developer enters an API key beside the URL, and every request then carries Authorization: Bearer <key> — or a header you name, such as X-Api-Key. With no key entered, no such header is sent at all.

The caller's own SSO token

The platform can instead authenticate to you as whoever is signed in — the only mode that tells you who is asking, so an agent with its own per-user authorization can apply it. Accept the same realm and audience; tokens are re-minted mid-run, so cache on the claims, not the token text.

PART 1 · SCHEMA

Chat endpoint — the request

```
{
  "model": "default",
  "messages": [
    {"role": "user", "content": "What was ACME's
      outstanding balance at the end of Q2?"}
  ],
  "stream": false,
  "skill_studio": {
    "timeout_s": 115.0,
    "trace_data": {
      "trace_id":   "9f3e11c8a2b04d7e8c1f5a6b7d8e9f01",
      "session_id": "9f3e11c8a2b04d7e8c1f5a6b7d8e9f01",
      "user_id": "alice",
      "tags": ["eval_billing"]
    },
    "skills": {                          // optional
      "billing/SKILL.md": "---\nname: billing\n---# ..."
    }
  }
}
```

Field	What you do with it
messages	Exactly one user message. No system message — your prompt stays yours.
model / stream	Constants ("default", false). You may ignore both.
timeout_s	Your budget for this call. Honour it, clamp it to a ceiling of your own, and return 504 on expiry.
trace_id	Use the one we send as your Langfuse trace id. Mint your own and every trace-reading feature goes dark.
skills	Absent → your own files. A map → these files only, for this call. {} → no skill files at all.

Response — an ordinary chat completion; the answer is read from `choices[0].message.content`. Never return an empty answer or a tool-call-only message — that fails the question rather than scoring it. If you have nothing to say, return a 5xx with a reason.

PART 1 · SCHEMA

Skills endpoint — the response

```
{
  "version": "a1b2c3d",
  "skills": {
    "billing/SKILL.md":
      "---\nname: billing\ndescription: Invoices,
       balances and refunds.\n---\n# Billing ...",
    "billing/references/refunds.md":
      "# Refund rules\nProrated by service days.\n",
    "reporting/SKILL.md": "# Reporting ..."
  }
}
```

Four rules that matter

- Flat, not nested. One key per file, and walk every level — a skill is a directory, and its reference files matter as much as SKILL.md.
- Full text, never truncated. A developer edits this content and sends it back; a truncated file becomes a destructive edit.
- `{}` is a valid answer — an agent with no skills is a supported configuration. If you cannot read your own directory, return 5xx, never an empty map.
- version moves whenever your answers would move: a skill edit, a model swap, a prompt change, a redeploy. A constant is worse than nothing.

The skills override has three states — test for the key, not for truthiness

`key absent`

Use your own files, exactly as normal.

`a populated map`

Use these files and only these, for this one call. Replacement, not a patch — and never persisted.

`{}`

Use no skill files at all. “Does it still work without the skill?” is a question people deliberately ask.

Path keys are attacker-influenced strings: reject `..`, a leading `/`, backslashes, NUL bytes and empty keys with a 400 before they become filesystem paths.

PART 1 · THREE LEVELS

You do not have to build all of it

The platform works with what you give it, and tells you what you are missing rather than refusing to start.

1 **An OpenAI-compatible chat endpoint**

What most agents already have

YOU GET

Evaluation

Run an eval set and see the score, the answers and the judge's verdicts — plus the slice of the playground that only needs an answer.

2 **+ the skills endpoint**
+ honour the skills override

Two behaviours, one new route

YOU GET

The full playground

View and edit a copy of your skill files, ask one question at a time against the edit, plus skill-coverage warnings and staleness detection when your server moves underneath you.

3 **+ traces in Langfuse under the trace id we send**

Not a third endpoint

YOU GET

Optimization

Hundreds of rollouts against candidate versions of a skill, scored and compared, with a validation gate that discards the edits that did not help. The platform verifies both behaviours before a run starts.

Every level is checked before it is offered: the platform names what is missing rather than failing halfway through a run.

PART 1 · GETTING THERE

You probably should not write this by hand

The API reference is written to be handed to a machine: self-contained, no cross-references into our repo, with a reference implementation and an acceptance checklist at the end.

1 **Open the docs in the platform**

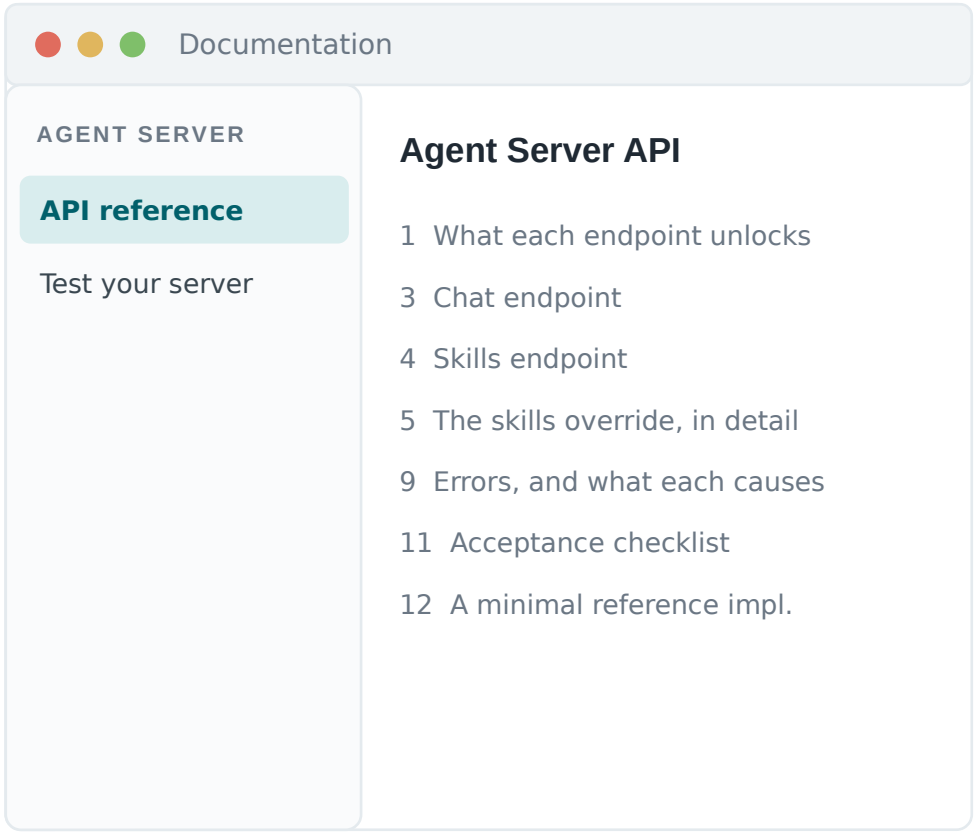
Agent Server → API reference. One page, 13 sections, everything the contract requires.

2 **Point your coding assistant at it**

That page is the whole spec — it can implement your server against it without reading our codebase.

3 **Test your server, in the platform**

The Test-your-server page sends a real question with a real skills override and tells you which level you have actually reached.



→ **DEMO: the docs page, then Test your server**

PART 2 · A SECOND USE

The same eval loop, pointed at skill selection

Evaluation was built to grade the answer. It grades a routing decision just as well — the trick is entirely in how the question is phrased.

ANSWER EVALUATION

“What will TSMC's capacity be next year?”

Graded on: the answer the agent produced.

→

ROUTING EVALUATION

“Which skill would you pick to answer the following question: What will TSMC's capacity be next year?”

Graded on: the skill the agent names.

What that buys us

No new feature

Same upload, same run, same judge, same failure diagnosis. Routing became measurable without a line of new evaluation code.

The label was already there

Every eval question already carries a skill field — the skills it should have been answered with. That is the ground truth a routing question needs.

A cheaper signal

The agent stops before it works. One decision per question instead of a full trajectory, so a routing set runs fast and costs little.

PART 2 · IN FLIGHT

Routing optimization: why SkillOpt did not transfer

WHAT WE TRIED

Isolated mode's algorithm, pointed at the description

Small minibatches, an analyst call each, then a merge. It does not converge: the description keeps being rewritten and the score keeps moving without settling.

Optimizing a skill body is a large model: thousands of words, an edit appends to one section, and the loss follows the gradient down. A description is two or three sentences — a representation space so small that every edit is a rewrite, so the loss jumps across the surface instead of descending it.

WHAT WE DO NOW

One larger, stratified batch per step

- A big batch gives a more global gradient than a handful of questions can.
- Stratified sampling interleaves the skills, so every description is rewritten from evidence about itself — not dragged by whichever skill happened to turn up.
- One analyst call over the whole batch: nothing left for a merge stage to pick between with the evidence already discarded.

The context problem a big batch creates

Hundreds of full traces do not fit in the optimizer's context window. So the step sends a digest instead: a per-skill confusion matrix of what was tagged versus what was opened, plus the agent's own setup, folded once and marked where it varied.

! Still in flight

The experiments are running now — no convergence numbers to show yet. What is settled is the diagnosis and the shape of the fix; the results land next sprint.

WHAT I'D LIKE FROM YOU

Start at level 1. It is smaller than it looks.

1 **Bring the chat endpoint you already have**

If it speaks OpenAI chat completions, honour timeout_s and reuse our trace id, you can be evaluated this week.

2 **Hand the API reference to your coding assistant**

It is written for exactly that: self-contained, with a reference implementation and a curl acceptance checklist.

3 **Prove it with Test your server**

Before you wire up a run — it tells you which of the three levels you have actually reached.

4 **Tell me where the doc made you guess**

Ambiguity in that page is a bug in that page, and it is the cheapest thing on this list for me to fix.

Questions?